

REPORTE TÉCNICO FINAL

Monitor y Simulador Benigno de Comportamiento

UANL

Producto Integrador de Aprendizaje (PIA)

Programación Avanzada para Ciberseguridad

Integrante	Matrícula / Rol
Brandon Yahir Flores Garcia	2037084
Jonathan Emir Jacobo Martinez	2053285
Jose Miguel Castellanos Martinez	2064175
Juan Carlos Fernandez Flores	2133011

Actualmente, el panorama de la ciberseguridad necesita medidas proactivas capaces de detectar comportamientos anormales exactamente cuando ocurren. Los métodos de detección tradicionales basados en firmas estáticas o en la interceptación de bibliotecas de espacio de usuario tienen una alta fragilidad frente a amenazas más avanzadas que despliegan técnicas de evasión, inyección de procesos y suplantación de metadatos contra estos. En este sentido, el análisis de comportamiento en tiempo real y la ingeniería inversa binaria se consolidan como habilidades fundamentales para el analista de seguridad contemporáneo.

Objetivo del Proyecto

Crear dos módulos de un sistema en C++ para Linux juntos:

- **Monitor Defensivo:** Identifica comportamientos sospechosos en el momento en que se detectan mediante el listado de procesos en el sistema de archivos virtual (/proc), verificación estática de binarios ELF, determinación de hash SHA-256 y correlación de riesgos puntuados.
- **Simuladores Ofensivos Benignos:** Un conjunto de binarios (stager, inject, spoof, target) desarrollados en configuraciones sin símbolos de depuración y con símbolos de depuración para simular comportamientos maliciosos controlados (enlaces C2, manipulación de memoria y manipulación de identidad) para demostrar que el monitor funciona, pero sin causar daño físico al sistema.

La parte defensiva aísla e identifica procesos sospechosos, mientras tanto el componente ofensivo se somete a un análisis profundo utilizando herramientas de ingeniería inversa. Este proceso holístico de análisis de procesos, ingeniería inversa estática, análisis dinámico y auditoría de seguridad muestra el dominio técnico de las Fases I–IV del curso, donde las herramientas de detección más efectivas deben construirse con los niveles más bajos del sistema operativo.

2 Resumen

Se desarrolló un sistema compuesto por dos subsistemas integrados: un monitor defensivo simulador y cuatro simuladores ofensivos benignos. Todos los componentes compilan en una sola instrucción make all y generan binarios duales: uno con símbolos de depuración _sys y otro ofuscado _raw o stripped para ejercicios de ingeniería inversa.

Componente	Función	Resultado
simulador	Monitor defensivo en tiempo real	Detecta y audita comportamiento anómalo en RAM y disco.
Stager	Conexión TCP 127.0.0.1:8080 (C2)	Score 80/100 — Nivel ALTO
inject	Inyección de código vía ptrace	Score 45/100 — Nivel MEDIO
spooof	Falsificación /proc/self/comm	Falsificación superficial demostrada
target	Proceso víctima para inyección	Línea base operativa

3 Diseño del Sistema

Arquitectura modular: El subsistema defensivo (simulador) sigue un diseño en capas desacopladas donde cada módulo procesa un vector de telemetría específico antes de consolidar el diagnóstico de seguridad:

Capa	Módulo	Responsabilidad
Recolección	enumerator.cpp	Escanea /proc para listar PIDs activos y extraer sus metadatos básicos
Capa	Módulo	Responsabilidad
Análisis	elf_analyzer.cpp	Inspecciona cabeceras ELF, secciones y símbolos importados del ejecutable en disco.
Hashing	hash_engine.cpp	SHA-256 de ejecutable en disco vía OpenSSL EVP
Correlación	scorer.cpp	Pondera IOCs: socket, ptrace, spoofing, secciones sospechosas
Persistencia	logger.cpp	Guarda el registro de los eventos en logs/events.log con timestamp.
Orquestador	main.cpp	Loop principal: enumerar → analizar → puntuar → registrar

Flujo de ejecución del monitor: El orquestador ejecuta un ciclo continuo de escaneo. Para cada proceso activo en /proc, el sistema lee sus metadatos, localiza el ejecutable vía /proc/[pid]/exe , ejecuta el análisis de las cabeceras ELF y calcula su firma criptográfica SHA-256. Si el sistema identifica indicadores anómalos, detona una auditoría heurística profunda directamente en la memoria RAM del proceso . Tras la evaluación, si el Scorer determina un nivel de riesgo MEDIO o ALTO (Score ≥ 45), el orquestador emite una advertencia en consola bajo la etiqueta

[ALERT] y consolida el reporte forense en el archivo logs/events.log

4 Implementación

4.1 Enumerador de procesos (enumerator.cpp)

Este módulo actúa como el sensor primario del monitor defensivo. Utiliza las funciones estándar de POSIX (opendir() y readdir()) para iterar sobre la raíz del sistema de archivos virtual /proc. Para garantizar la eficiencia del escaneo, aplica un filtro matemático (isNumeric) que descarta carpetas genéricas y aísla únicamente los directorios correspondientes a PIDs activos.

Por cada proceso válido, el módulo extrae dos identificadores cruciales:

1. **Identidad Nominal:** Leyendo directamente el archivo /proc/[pid]/comm.
2. **Ruta Absoluta:** Resolviendo el enlace simbólico del binario en disco a través de la llamada al sistema readlink("/proc/[pid]/exe").

Filtrado Heurístico : La función FindSuspicious cruza los nombres obtenidos contra un arreglo predefinido de firmas (SUSPICIOUS_NAMES), el cual alerta sobre términos como "stager", "inject" y "rat". El motor utiliza std::string::find para buscar estas palabras como subcadenas dentro del nombre completo del proceso.

```
=====
Procesos analizados: 258 Alertas generadas: 6 Log guardado en: logs/events.log
=====
```

4.2 Analizador ELF estático (elf_analyzer.cpp)

Este módulo es el motor de inspección profunda del sistema y opera en dos fases de validación arquitectónica:

Fase Estática (Disco): Parsea las cabeceras del binario utilizando las definiciones nativas <elf.h>. Su validación principal ocurre en la tabla de símbolos dinámicos (SHT_DYNSYM), donde escanea las funciones importadas buscando dos vectores de riesgo: capacidades de red para Comando y Control (socket, bind, listen) y llamadas al sistema para secuestro de memoria (ptrace, mmap, execve).

Fase Dinámica: Para detectar amenazas inyectadas que no residen en disco, el algoritmo lee /proc/[pid]/maps para aislar regiones de memoria con permisos de ejecución (r-xp). Posteriormente, accede a /proc/[pid]/mem y realiza un escaneo lineal buscando firmas binarias anómalas (ej. el patrón YARA 0xeb, 0xfe que indica un loop infinito inyectado)

```
// Símbolos [*] [EDR] Iniciando escaneo heurístico de RAM en PID: 7366...
[!!!!] INCIDENTE CRÍTICO: Ejecución anómala detectada en RAM.
-> Coordenada comprometida: 0x7f01318f06ef
-> Patrón validado: EB FE (Loop Infinito)
```

4.3 Motor de hashing SHA-256 (hash_engine.cpp)

Este módulo genera firmas criptográficas inmutables para la identificación unívoca de binarios, permitiendo la correlación contra bases de datos de Indicadores de Compromiso (IoCs) independientemente de si el atacante altera el nombre del archivo.

El algoritmo abre el ejecutable en modo binario e implementa un búfer de lectura en bloques de 8192 bytes. Utiliza la API nativa de la librería OpenSSL (SHA256_Init, SHA256_Update, SHA256_Final) para procesar el flujo de datos sin cargar el archivo completo en RAM, previniendo el agotamiento de recursos del sistema al analizar *payloads* de gran tamaño. Finalmente, formatea el *digest* resultante en una cadena hexadecimal de 64 caracteres. Por diseño arquitectónico, si la ruta apunta a un proceso nativo del kernel sin representación física en disco, el módulo aborta la lectura de forma pasiva, evitando interrupciones en el orquestador.

```
PROCESO SOSPECHOSO DETECTADO
=====
PID: 7366
Nombre: stager_sys
Ruta: /home/brandon/Escritorio/PIA_PAC_TEAM2-
main/PIA_PAC_TEAM2/build/stager_sys
[2026-05-22 02:42:49] [ALERT] Proceso sospechoso: stager_sys [PID: 7366]
[2026-05-22 02:42:49] [INFO] Calculando SHA256 de:
/home/brandon/Escritorio/PIA_PAC_TEAM2-main/PIA_PAC_TEAM2/build/stager_sys
SHA256: 6bd4a155e2a86e38ac37e010e0f51482cb14ca8fd2e230861a97d29cb9f7615b
[2026-05-22 02:42:49] [INFO] SHA256:
6bd4a155e2a86e38ac37e010e0f51482cb14ca8fd2e230861a97d29cb9f7615b
[2026-05-22 02:42:49] [INFO] Analizando ELF: /home/brandon/Escritorio/PIA_PAC_TEAM2-
main/PIA_PAC_TEAM2/build/stager_sys
```

4.4 Scoring de riesgo (scorer.cpp)

Este módulo centraliza la toma de decisiones evaluando la telemetría recolectada por el enumerador y el analizador ELF. Implementa un algoritmo de puntuación aditiva y determinista que calcula un índice de riesgo acotado entre 0 y 100 puntos.

La arquitectura de la función Evaluate asigna pesos matemáticos estrictos a los Indicadores de Compromiso (IoCs):

- **Análisis de Metadatos:** Penaliza con +30 puntos una coincidencia directa de nombre malicioso y con +15 puntos la coincidencia por subcadenas (ej. patrones como "stager", "inject" o "rat").
- **Capacidades Binarias (ELF):** Suma +25 puntos si el ejecutable importa símbolos de red, +20 puntos por símbolos de ejecución en memoria, y +10 puntos si contiene secciones dinámicas notables.

El sistema clasifica la amenaza evaluando el puntaje final contra umbrales rígidos: BAJO (0-39), MEDIO (40-69) y ALTO (70-100). Esta correlación justifica matemáticamente por qué un proceso del sistema mal filtrado (migration/0) alcanza solo un Nivel MEDIO (45 puntos), mientras que el stager, al sumar los vectores de red y secciones, alcanza el Nivel ALTO (80 puntos).

```
PID: 7424
Nombre: inject_sys
Ruta: (no accesible)
[2026-05-22 02:42:49] [ALERT] Proceso sospechoso: inject_sys [PID: 7424]
[2026-05-22 02:42:49] [INFO] Ejecutando auditoria profunda en RAM para PID: 7424
[-] [Monitor] Error: No se pudo acceder a la RAM del PID 7424 (Requiere sudo)
>>> SCORE DE RIESGO: 45/100 [MEDIO]
>>> Razones: nombre sospechoso (+30); coincide con patron 'inject' (+15);
```

5 Pruebas Realizadas

Se diseñó una suite de pruebas con 4 escenarios que cubren el flujo completo de detección. Cada prueba se ejecutó tres veces para validar consistencia.

ID	Escenario	Score Obtenido	Resultado
T-01	target solo (sin amenaza)	< 40/100	BAJO — OK
T-02	stager activo (socket C2)	80/100	ALTO — Detectado
T-03	inject sobre target	45/100	MEDIO — Detectado
T-04	spoof de /proc/self/comm	< 40/100	BAJO — Detectado

Logs generados: Cada evento queda registrado en logs/events.log con el formato: [YYYY-MM-DD HH:MM:SS] [LEVEL] PID=<pid> COMM=<nombre> SCORE=<n> PATH=<ruta>.

Para el reconocimiento inicial de los artefactos, se utilizó la herramienta nativa `file` para auditar la estructura de los ejecutables sin detonar su carga útil.

- Validación de Formato: El comando `file` permitió confirmar que tanto los binarios de prueba (`inject_sys_stripped`) como los de ataque (`inject_raw_stripped`) son ejecutables de tipo ELF 64-bit LSB pie executable para arquitectura x86-64.
- Evidencia de Ofuscación: La etiqueta "stripped" (presente en ambas capturas) confirma que los archivos han sido procesados para eliminar las tablas de símbolos y la información de depuración. Esto demuestra un esfuerzo técnico por ocultar la lógica interna y las referencias a funciones del sistema, obligando al analista a recurrir a técnicas de ingeniería inversa (como el uso de Ghidra) para comprender el flujo de control.
- Integridad: La comparación de los `BuildID[sha1]` únicos para cada muestra permitió verificar que, a pesar de compartir funcionalidad, son binarios distintos, garantizando la trazabilidad forense de cada variante.

```
carlinhos@kdot:~/PIA_PAC_TEAM2-main/PIA_PAC_TEAM2$ file bin/inject_sys_stripped
bin/inject_sys_stripped: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=b795f86fe8228691093008854be3ad8b4ac88ee2, for GNU/Linux 3.2.0, stripped
carlinhos@kdot:~/PIA_PAC_TEAM2-main/PIA_PAC_TEAM2$
```

```
carlinhos@kdot:~/PIA_PAC_TEAM2-main/PIA_PAC_TEAM2$ file bin/inject_raw_stripped
bin/inject_raw_stripped: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=6e521d2e05e531f54c6881a29485b60f1d0a09b9, for GNU/Linux 3.2.0, stripped
carlinhos@kdot:~/PIA_PAC_TEAM2-main/PIA_PAC_TEAM2$
```

El análisis dinámico se realizó mediante la detonación controlada de los binarios en un entorno aislado, priorizando la intercepción de llamadas al sistema y la observación directa de la memoria viva.

- **Auditoría de Librerías (ltrace):** Se interceptó la ejecución del inyector para identificar sus dependencias en tiempo real. La traza obtenida confirma el uso de bibliotecas de la libc y libstdc++, revelando el Name Mangling de C++ (_ZStls...) durante el manejo de entrada/salida previo al ataque. Esto demuestra cómo el binario opera bajo la infraestructura legítima del sistema antes de ejecutar su carga útil.
- **Intercepción a Nivel Kernel (strace):** El rastreo de llamadas al sistema documentó la ejecución material del secuestro. La traza evidencia la secuencia: PTRACE_ATTACH (secuestro del PID 5782), PTRACE_GETREGS (extracción de punteros de instrucción), y PTRACE_POKETEXT, capturando el instante preciso en que se inyecta el shellcode en la memoria del objetivo.

```
carlinhos@kdot:~/SecurityTools$ cd
carlinhos@kdot:~$ cd PIA_PAC_TEAM2-main/
carlinhos@kdot:~/PIA_PAC_TEAM2-main$ cd PIA_PAC_TEAM2/
carlinhos@kdot:~/PIA_PAC_TEAM2-main/PIA_PAC_TEAM2$ sudo strace -e trace=ptrace ./build/inject_sys 5583
[*] Iniciando secuestro del proceso: 5583
ptrace(PTRACE_ATTACH, 5583) = 0
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_TRAPPED, si_pid=5583, si_uid=1000, si_status=SIGSTOP, si_utime=0, si_stime=0} ---
[+] Proceso interceptado exitosamente.
ptrace(PTRACE_GETREGS, 5583, {r15=0x55dbd3d37dc8, r14=0x7fa77f0fa000, r13=0x7ffd939b4b88, r12=0, rbp=0, rbx=0x7fa77f06f780, r11=0x202, r10=0x7ffd939b4a10, r9=0, r8=0, rax=0xffffffffffffdfc, rcx=0x7fa77ec9b687, rdx=0x7ffd939b4a10, rsi=0, rdi=0, orig_rax=0xe6, rip=0x7fa77ec9b687, cs=0x33, eflags=0x202, rsp=0x7ffd939b49d0, ss=0x2b, fs_base=0x7fa77f06f780, gs_base=0, ds=0, es=0, fs=0, gs=0}) = 0
[+] RIP actual: 0x7fa77ec9b687
ptrace(PTRACE_POKETEXT, 5583, 0x7fa77ec9b687, 0x55050f3cb0ff3148) = 0
[+] Liberando proceso. El flujo de ejecución ha sido comprometido.
ptrace(PTRACE_DETACH, 5583, NULL, 0) = 0
[*] Presiona ENTER para finalizar el inyector...
```

```
carlinhos@kdot:~/PIA_PAC_TEAM2-main/PIA_PAC_TEAM2$ sudo ltrace ./build/inject_sys 5782
_ZNSt7_cxx112basic_stringIcSt11char_traitsIcESaIcEEC1EPKcRKS3_(0x7fffc26868f0, 0x7fffc26876b6, 0x7fffc268691f, 0x7fffc26876b6) = 0x7fffc2686900
_ZNKS7_cxx112basic_stringIcSt11char_traitsIcESaIcEE5c_strEv(0x7fffc26868f0, 0, 10, 0x32383735) = 0x7fffc2686900
_errno_location() = 0
_errno_location() = 0
_errno_location() = 0
_errno_location() = 0
_errno_location() = 0
_ZNSt7_cxx112basic_stringIcSt11char_traitsIcESaIcEE1Ev(0x7fffc26868f0, 5782, 0, 0x7fffc2686904) = 0x7fffc2686900
_ZStlsISt11char_traitsIcEESt13basic_ostreamIcT_ES5_PKc(0x564cb79aa0c0, 0x564cb79a8038, 0, 0x7fffc2686904) = 0x564cb79aa0c0
_ZNSolsEi(0x564cb79aa0c0, 5782, 0x564cb79aa0c0, 1024) = 0x564cb79aa0c0
_ZStlsISt11char_traitsIcEESt13basic_ostreamIcT_ES5_PKc(0x564cb79aa0c0, 0x564cb79a805e, 0x564cb79aa0c0, 0x32383735[*] Iniciando secuestro del proceso: 5782) = 0x564cb79aa0c0
ptrace(16, 5782, 0, 0 <no return ...> = 0
--- SIGCHLD (El proceso hijo terminó) --- = 5782
<... ptrace resumed> ) = 5782
wait(nil) = 5782
_ZStlsISt11char_traitsIcEESt13basic_ostreamIcT_ES5_PKc(0x564cb79aa0c0, 0x564cb79a8098, 61, 5782[+] Proceso interceptado exitosamente. = 0
) = 0x564cb79aa0c0
ptrace(12, 5782, 0, 0x7fffc2686810) = 0
_ZStlsISt11char_traitsIcEESt13basic_ostreamIcT_ES5_PKc(0x564cb79aa0c0, 0x564cb79a80c0, 0, 0x7f327b7198c8) = 0x564cb79aa0c0
_ZNSolsEPFRSt8ios_baseS0_E(0x564cb79aa0c0, 0x564cb79a761c, 0x564cb79aa0c0, 0x7f327b7f25c0) = 0x564cb79aa0c0
_ZNSolsEy(0x564cb79aa0c0, 0x7f4c4889b687, 0x564cb79aa0c0, 0x7f327b7f25c0) = 0x564cb79aa0c0
_ZStlsISt11char_traitsIcEESt13basic_ostreamIcT_ES5_PKc(0x564cb79aa0c0, 0x564cb79a805e, 0x564cb79aa0c0, 0x3238366239383834[+] RIP actual: 0x7f4c4889b687) = 0x564cb79aa0c0
ptrace(4, 5782, 0x7f4c4889b687, 0x55050f3cb0ff3148) = 0
_ZStlsISt11char_traitsIcEESt13basic_ostreamIcT_ES5_PKc(0x564cb79aa0c0, 0x564cb79a80d8, 0, 0x7f327b7198c8[+] Liberando proceso. El flujo de ejecución ha sido comprometido. = 0
) = 0x564cb79aa0c0
ptrace(17, 5782, 0, 0) = 0
_ZStlsISt11char_traitsIcEESt13basic_ostreamIcT_ES5_PKc(0x564cb79aa0c0, 0x564cb79a8120, 0, 0x7f327b7198c8) = 0x564cb79aa0c0
_ZNSolsEPFRSoS0_E(0x564cb79aa0c0, 0x7f327b940800, 0x7f327b940800, 0x7f327b7f25c0[*] Presiona ENTER para finalizar el inyector... = 0x564cb79aa0c0
) = 0x564cb79aa0c0
_ZNSi3getEv(0x564cb79aa0c0, 0, 0x7f327ba5c310, 0x7f327b7f25c0 <no return ...>
```

Esta sección documenta la disección técnica del inyector (inject_sys y su variante inject_raw_stripped) mediante el uso de Ghidra, permitiendo visualizar cómo la lógica de ejecución fuerza el comportamiento malicioso a nivel de arquitectura de procesador.

- **Descompilación de Control de Flujo:** A través del grafo de funciones en real_main, se analizó la estructura de control del inyector. El código descompilado confirma la invocación secuencial de las APIs de depuración de Linux (ptrace(PTRACE_ATTACH, ...) y ptrace(PTRACE_GETREGS, ...)) para usurpar el control del proceso víctima. Esta lógica se mantiene consistente tanto en la versión con símbolos como en la versión stripped, demostrando que el despojo de metadatos no oculta la intención operativa del atacante.

```
Decompile: real_main - (inject_sys_stripped)
4{
5    ostream *poVar1;
6    undefined8 uVar2;
7    long ptrace_status;
8    undefined1 registro_cpu [128];
9    ulonglong local_b8;
10   string local_58 [47];
11   allocator local_29;
12   allocator *local_28;
13   uint target_pid;
14
15   if (argc == 2) {
16       local_28 = &local_29;
17       /* try { // try from 001012c1 to 001012c5 has its CatchHandler @ 001014c3 */
18       std::_cxx11::string::string(local_58, (char *)argv[1], &local_29);
19       /* try { // try from 001012d7 to 001012db has its CatchHandler @ 001014b2 */
20       target_pid = FUN_001014d4(local_58, 0, 10);
21       std::_cxx11::string::string::string(local_58);
22       FUN_001017aa(&local_29);
23       poVar1 = std::operator<<((ostream *)std::cout, "[*] Iniciando secuestro del proceso: ");
24       poVar1 = (ostream *)std::operator<<(poVar1, target_pid);
25       std::operator<<(poVar1, "\n");
26       ptrace_status = ptrace(PTRACE_ATTACH, (ulong)target_pid, 0, 0);
27       if (ptrace_status < 0) {
28           std::operator<<((ostream *)std::cerr, "[X] Fallo ptrace_attach. Verifique permisos YAMA.\n");
29           uVar2 = 1;
30       }
31       else {
32           wait((void *)0x0);
33           std::operator<<((ostream *)std::cout, "[+] Proceso interceptado exitosamente.\n");
34           ptrace(PTRACE_GETREGS, (ulong)target_pid, 0, registro_cpu);
35           poVar1 = std::operator<<((ostream *)std::cout, "[+] RIP actual: 0x");
36           poVar1 = (ostream *)std::operator<<(poVar1, FUN_0010161c);
```

- **Localización de Carga Útil (Shellcode):** Mediante el escaneo de memoria en Ghidra, se aisló el payload malicioso contenido en el segmento de datos. Se localizó la secuencia de opcodes 0F 05 (el syscall que invoca la salida del sistema) incrustada dentro de la estructura de datos del ejecutable.

		LAB_0010200e	
0010200e	48 31 ff	XOR	RDI, RDI
00102011	b0 3c	MOV	AL, 0x3c
00102013	0f 05	SYSCALL	

- **Análisis de Punto de Entrada:** La inspección del ensamblador en el punto de entrada (`__libc_start_main`) demuestra cómo el binario inicializa su entorno antes de transferir el control a la rutina `real_main`.

```

10114f 45 31 c0      XOR     R8D,R8D
101152 31 c9        XOR     ECX,ECX
101154 48 8d 3d      LEA     RDI,[FUN_00101229]
          ce 00 00 00
10115b ff 15 5f      CALL    qword ptr [-><EXTERNAL>:__libc_start_main]  undefined __libc_start_main()
          2e 00 00                                = 00105030
101161 f4          HI T

00101154 48 8d 3d      LEA     RDI,[real_main]
          ce 00 00 00
saved 0 ff 15 5f      CALL    qword ptr [-><EXTERNAL>:__libc_start_main]  undefined __libc_start_main()
          2e 00 00                                = 00105030

```

9 Riesgos y Mitigaciones

Aunque el proyecto opera en un entorno controlado y los simuladores son benignos, se identificaron riesgos técnicos y operacionales que se documentan junto con las mitigaciones implementadas o recomendadas.

Riesgo	Probabilidad	Impacto	Mitigación
Manipulación de procesos mediante ptrace	Alta	Alto	Auditoría de kernel para detectar la secuencia de llamadas al sistema que caracterizan al inyector
Evasión de detección mediante spoofing	Media	Medio	Validación cruzada resolviendo el enlace real del binario para ignorar nombres falsos
Comando y Control de red	Alta	Alto	Identificación estática de capacidades de red en las cabeceras del binario
Ofuscación de código mediante strip	Media	Medio	Análisis híbrido combinando desensamblado manual y rastreo dinámico de ejecución

La seguridad debe entenderse de forma integral. Comprobamos que un sistema no se protege bloqueando una sola puerta. El proyecto demostró que un atacante tiene múltiples formas de operar: puede esconder el nombre de un proceso, inyectar código en la memoria de otro programa o abrir canales de comunicación ocultos. Por esto, una única medida de defensa es insuficiente y se requiere una arquitectura de monitoreo que cubra distintas capas del sistema.

Por otro lado, la ofuscación no es un obstáculo definitivo para un analista. Aunque aplicamos técnicas para borrar los nombres de las funciones y limpiar los ejecutables, el comportamiento real de los programas siempre deja pistas que no se pueden ocultar. Sin importar qué tan limpio esté el archivo en el disco, la forma en que interactúa con el sistema operativo y con la memoria del equipo permite identificarlo y perfilarlo con precisión.

Finalmente, la lección principal es que la mejor estrategia de defensa es no depender de una sola herramienta. Al analizar los programas tanto en reposo, revisando su estructura interna, como en acción, observando cómo operan en tiempo real, logramos entender perfectamente sus planes. Esa combinación de métodos es lo que realmente marca la diferencia entre un sistema vulnerable y uno capaz de detectar amenazas avanzadas.

El sistema fue ejecutado exclusivamente en entorno de laboratorio aislado. Todos los simuladores generan comportamiento benigno y no realizan modificaciones permanentes al sistema ni conexiones fuera del loopback (127.0.0.1).